

Contents

- CDI 1

Technical Documentation - Clinical Documentation Improvement System 1

. 1

1. (System Architecture) 1

. 2

Frontend Stack 2

Backend Stack 4

. 5

1. (Authentication) 5

2. (Encryption) 6

3. Rate Limiting 6

4. Input Validation 7

. 7

Google Gemini Integration 7

API Key Rotation 8

. 8

MongoDB Schema 8

Indexes 9

. 9

Prometheus Metrics 9

Logging 10

. 10

. 10

Environment Variables 11

. 11

. 11

. 11

- CDI

Technical Documentation - Clinical Documentation Improvement System

1. (System Architecture)

(User)
(Browser - Chrome/Safari/Edge)

HTTPS/TLS 1.3

Frontend - React.js 18

- UI Components (Shadcn UI + Tailwind CSS)
- Login / Register / MFA
 - Dashboard
 - Note Analysis
 - Chat Interface
 - Admin Panel

- State Management (React Context)
- Auth Context
 - Language Context (i18n)
 - Toast Notifications (Sonner)

REST API (JSON)

Backend - FastAPI (Python)

API Endpoints

- /api/auth/* (Authentication)
- /api/notes/* (Notes Management)
- /api/analysis/* (AI Analysis)
- /api/chat/* (AI Chat)
- /api/admin/* (Admin Functions)

Security Layer

- JWT Authentication
- Rate Limiting (SlowAPI)
- CORS Protection
- Input Validation (Pydantic)

Business Logic

- User Management
- Note Processing
- AI Integration
- Audit Logging

MongoDB (Database)	Google Gemini AI (API Service)
-----------------------	-----------------------------------

Collections:	- Text Analysis
- users	- ICD-10-CM
- notes	- Queries Gen
- analyses	- Chat
- otp_records	
- audit_logs	
- messages	

Frontend Stack

1. React.js Application

```
//
{
  "react": "^18.2.0",
  "react-dom": "^18.2.0",
  "react-router-dom": "^6.20.0",
  "axios": "^1.6.2",
  "i18next": "^23.7.6",
  "react-i18next": "^13.5.0"
}

// UI Framework
{
  "@radix-ui/react-*": "^1.0.0", // Shadcn UI Components
  "tailwindcss": "^3.3.0",
  "lucide-react": "^0.294.0"
}

// State & Notifications
{
  "sonner": "^1.2.0", // Toast notifications
  "zustand": "^4.4.7" // State management
}
```

2.

```
frontend/src/
components/
  ui/                # Shadcn UI Components
    button.jsx
    card.jsx
    input.jsx
    dialog.jsx
    ...
  Navbar.jsx        # Navigation bar
  Footer.jsx        # Footer component
  ChatWidget.jsx    # AI Chat widget
contexts/
  LanguageContext.jsx # i18n context
pages/
  Login.jsx         # Login page
  Register.jsx      # Registration
  MFAVerification.jsx # MFA OTP
  Dashboard.jsx     # User dashboard
  NewNote.jsx       # Create note
  Analysis.jsx       # View analysis
  Chat.jsx           # AI Chat
  ChatEnhanced.jsx  # Enhanced chat
  AdminDashboard.jsx # Admin panel
  ...
utils/
  errorHandler.js   # Error handling
App.js              # Main app component
index.js            # Entry point
i18n.js             # i18n configuration
```

Backend Stack

1. FastAPI Application

```
#
fastapi==0.104.1
uvicorn==0.24.0
pydantic==2.5.0
python-multipart==0.0.6

# Database
motor==3.3.2          # MongoDB async driver
pymongo==4.6.0

# Security
python-jose==3.3.0    # JWT
passlib==1.7.4        # Password hashing
bcrypt==4.1.1
python-dotenv==1.0.0

# AI Integration
google-generativeai==0.3.1

# Email
aiosmtplib==3.0.1

# Monitoring
prometheus-client==0.19.0
slowapi==0.1.9        # Rate limiting
```

2.

```
backend/
server.py          # Main FastAPI app
security_models.py # Pydantic models
security_utils.py  # Security utilities
security_routes.py # Auth endpoints
clinical_questions.py # Clinical Q&A
migration_update_users.py # DB migrations
requirements.txt   # Dependencies
.env               # Environment vars
```

3. Endpoints Authentication (/api/auth/*):

```
POST /api/auth/register      #
POST /api/auth/login-step1   # ( 1)
POST /api/auth/login-step2   # MFA OTP ( 2)
POST /api/auth/logout        #
POST /api/auth/forgot-password #
POST /api/auth/reset-password #
```

Notes Management (/api/notes/*):

```
POST /api/notes/create      #
GET /api/notes/user         #
GET /api/notes/{note_id}    #
PUT /api/notes/{note_id}    #
DELETE /api/notes/{note_id} #
```

AI Analysis (/api/analysis/*):

```

POST /api/analysis/analyze          #
GET  /api/analysis/{analysis_id}    #
POST /api/analysis/reanalyze        #

```

Chat (/api/chat/*):

```

POST /api/chat/message              #
GET  /api/chat/history/{analysis_id} #
POST /api/chat/clinical-question    #

```

Admin (/api/admin/*):

```

GET  /api/admin/users                #
POST /api/admin/users/{user_id}/activate #
POST /api/admin/users/{user_id}/deactivate #
GET  /api/admin/statistics            #
GET  /api/admin/audit-logs            #

```

1. (Authentication)

JWT (JSON Web Tokens)

```

# Token
def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(hours=24)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(
        to_encode,
        SECRET_KEY,
        algorithm="HS256"
    )
    return encoded_jwt

# Token
def verify_token(token: str):
    try:
        payload = jwt.decode(
            token,
            SECRET_KEY,
            algorithms=["HS256"]
        )
        return payload
    except JWTError:
        raise HTTPException(401, "Invalid token")

```

MFA (Multi-Factor Authentication)

```

# OTP
def generate_otp() -> str:
    return ''.join(random.choices('0123456789', k=6))

# OTP
async def verify_otp(email: str, otp_code: str):
    otp_record = await db.otp_records.find_one({
        "email": email,

```

```

        "otp_code": otp_code,
        "is_used": False,
        "expires_at": {"$gt": datetime.now(timezone.utc)}
    })
    return otp_record is not None

```

2. (Encryption)

```

import bcrypt

#
def hash_password(password: str) -> str:
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(
        password.encode('utf-8'),
        salt
    )
    return hashed.decode('utf-8')

#
def verify_password(password: str, hashed: str) -> bool:
    return bcrypt.checkpw(
        password.encode('utf-8'),
        hashed.encode('utf-8')
    )

from cryptography.fernet import Fernet

#
def encrypt_data(data: str, key: bytes) -> str:
    f = Fernet(key)
    encrypted = f.encrypt(data.encode())
    return encrypted.decode()

#
def decrypt_data(encrypted: str, key: bytes) -> str:
    f = Fernet(key)
    decrypted = f.decrypt(encrypted.encode())
    return decrypted.decode()

```

3. Rate Limiting

```

from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)

# Rate Limiting
@app.post("/api/auth/login-step1")
@limiter.limit("5/minute") # 5
async def login(credentials: UserLogin):
    # ...
    pass

```

4. Input Validation

```
from pydantic import BaseModel, EmailStr, validator

class UserRegister(BaseModel):
    email: EmailStr #
    password: str
    full_name: str

    @validator('password')
    def validate_password(cls, v):
        #
        if len(v) < 8:
            raise ValueError('Password too short')
        if not re.search(r'[A-Z]', v):
            raise ValueError('Must contain uppercase')
        if not re.search(r'[a-z]', v):
            raise ValueError('Must contain lowercase')
        if not re.search(r'[0-9]', v):
            raise ValueError('Must contain digit')
        if not re.search(r'[@#$$%^&*]', v):
            raise ValueError('Must contain special char')
        return v
```

Google Gemini Integration

```
import google.generativeai as genai

# API
genai.configure(api_key=GEMINI_API_KEY)

# AI
model = genai.GenerativeModel('gemini-2.0-flash-exp')

#
async def analyze_clinical_note(note_text: str, language: str):
    prompt = f"""
        (CDI).
        :

        {note_text}

        :
        1.
        2. ICD-10-CM
        3.
        4. CDI

        JSON .
        """

    response = model.generate_content(prompt)
    result = json.loads(response.text)
    return result
```

API Key Rotation

```
#
GEMINI_API_KEYS = [
    os.environ['GEMINI_KEY_1'],
    os.environ['GEMINI_KEY_2'],
    os.environ['GEMINI_KEY_3']
]

def get_api_key():
    return random.choice(GEMINI_API_KEYS)
```

MongoDB Schema

1. Users Collection

```
{
  _id: ObjectId,
  id: UUID, // UUID
  email: String,
  password_hash: String,
  full_name: String,
  phone_number: String,
  role: String, // "admin" | "supervisor" | "user"
  mfa_enabled: Boolean,
  is_active: Boolean,
  created_at: ISODate,
  last_login: ISODate,
  failed_login_attempts: Number,
  account_locked_until: ISODate,
  supervisor_id: String //
}
```

2. Clinical Notes Collection

```
{
  _id: ObjectId,
  id: UUID,
  user_id: String,
  patient_id: String,
  patient_age: Number,
  patient_gender: String,
  specialty: String,
  diagnosis: String,
  note_text: String,
  created_at: ISODate,
  updated_at: ISODate
}
```

3. Analyses Collection

```
{
  _id: ObjectId,
  id: UUID,
  note_id: String,
```



```

    user_id: String,
    gaps: Array,
    icd10_codes: Array,
    physician_queries: Array,
    cdi_notes: String,
    created_at: ISODate,
    analysis_version: Number
}

```

4. Audit Logs Collection

```

{
  _id: ObjectId,
  user_id: String,
  action: String,
  resource: String,
  details: Object,
  ip_address: String,
  user_agent: String,
  timestamp: ISODate
}

```

Indexes

```

// Users
db.users.createIndex({ "email": 1 }, { unique: true })
db.users.createIndex({ "id": 1 }, { unique: true })
db.users.createIndex({ "role": 1 })

// Clinical Notes
db.clinical_notes.createIndex({ "user_id": 1 })
db.clinical_notes.createIndex({ "created_at": -1 })
db.clinical_notes.createIndex({ "id": 1 }, { unique: true })

// Analyses
db.analyses.createIndex({ "note_id": 1 })
db.analyses.createIndex({ "user_id": 1 })
db.analyses.createIndex({ "created_at": -1 })

// Audit Logs
db.audit_logs.createIndex({ "user_id": 1 })
db.audit_logs.createIndex({ "timestamp": -1 })
db.audit_logs.createIndex({ "action": 1 })

```

Prometheus Metrics

```

from prometheus_client import Counter, Histogram, Gauge

#
login_attempts = Counter('login_attempts_total', 'Login attempts')
api_requests = Counter('api_requests_total', 'API requests', ['endpoint', 'method'])

#
request_duration = Histogram('request_duration_seconds', 'Request duration')

```

```
#
active_users = Gauge('active_users', 'Active users')

Logging

import logging

# Logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('app.log'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger(__name__)

#
logger.info(f"User {user_id} logged in")
logger.error(f"Error analyzing note: {error}")
```

```
# Docker Compose
version: '3.8'

services:
  backend:
    build: ./backend
    ports:
      - "8001:8001"
    environment:
      - MONGO_URL=mongodb://mongo:27017
      - DB_NAME=clinical_doc_center
    depends_on:
      - mongo

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    environment:
      - REACT_APP_BACKEND_URL=https://api.cdi-system.sa

  mongo:
    image: mongo:7.0
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db
```

```
volumes:
  mongo_data:
```

Environment Variables

```
# Backend (.env)
```

```
MONGO_URL=mongodb://localhost:27017
DB_NAME=clinical_doc_center
SECRET_KEY=your-secret-key-here
GEMINI_API_KEY_1=your-gemini-key-1
GEMINI_API_KEY_2=your-gemini-key-2
GEMINI_API_KEY_3=your-gemini-key-3
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=587
SMTP_USERNAME=your-email@gmail.com
SMTP_PASSWORD=your-app-password
FRONTEND_URL=https://cdi-system.sa
```

```
# Frontend (.env)
```

```
REACT_APP_BACKEND_URL=https://api.cdi-system.sa
PORT=3000
```

-
1. **FastAPI:** <https://fastapi.tiangolo.com>
 2. **React.js:** <https://react.dev>
 3. **MongoDB:** <https://www.mongodb.com/docs>
 4. **Google Gemini:** <https://ai.google.dev>
 5. **Tailwind CSS:** <https://tailwindcss.com>
 6. **Shadcn UI:** <https://ui.shadcn.com>

1. **OWASP Top 10:** <https://owasp.org/www-project-top-ten>
2. **NIST Cybersecurity:** <https://www.nist.gov/cyberframework>
3. **HIPAA:** <https://www.hhs.gov/hipaa>

** 2025 CDI System - Technical Documentation**